electronics

MDPI

Article

Hybrid CPU–GPU Cutting Simulation Using Position-Based Dynamics and Internal Shape-Preserving Constraints in Unity 3D

Lyudmila Khan [1], Yoo-Joo Choi [2] and Min Hong [3,*]

[1] Department of Software Convergence, Soonchunhyang University, Asan-si 31538, Republic of Korea; khanmila1024@gmail.com
[2] Department of AI Software Engineering, Seoul Media Institute Technology, Seoul 03925, Republic of Korea; yjchoi@smit.ac.kr
[3] Department of Computer Software Engineering, Soonchunhyang University, Asan-si 31538, Republic of Korea
* Correspondence: mhong@sch.ac.kr

Abstract

Cutting simulation is a key component of surgical simulators, requiring real-time performance, numerical stability, and physically plausible deformation. Achieving these properties is challenging because cutting combines collision detection, dynamic topology modification, and computationally intensive physical simulation. In this work, deformable objects are modeled using Position-Based Dynamics (PBD), while volumetric behavior is approximated through internal shape-preserving constraints (ISPCs) applied to triangular surface meshes. The primary contribution is a hybrid CPU–GPU cutting architecture in Unity 3D, in which the computationally expensive triangle–triangle intersection test is parallelized using compute shaders, surface rendering is performed directly from GPU-resident geometry buffers, and CPU–GPU synchronization overhead is minimized. Compared with our previous implementation, which parallelized only deformation and collision handling, the proposed framework additionally accelerates the intersection detection stage of cutting. The method was evaluated with single- and multiple-cut experiments on eight triangular meshes of increasing complexity (20–30,000 triangles). Seven of the eight models sustained interactive frame rates, and the cutting stage contributed negligibly to steady-state cost—below 0.5 ms on average even under repeated free cutting—with frame time being dominated instead by deformation and inter-part collision handling. Relative to our previous work, average frame rates improved by up to 677% over the slicing-based approach and up to 395% over the CPU-based ISPC implementation. Volume was maintained to within 0.4% of the initial volume for models of low geometric complexity. Two limitations were observed: for the high-resolution Armadillo model, mean frame cost was dominated by deformation and collision, while rare cut-finalization frames produced large worst-case spikes, keeping it below interactive rates; the Octopus model exhibited markedly reduced volume preservation, stabilizing at 73.4% of its initial volume. These results indicate that the proposed hybrid method substantially improves the efficiency of cutting simulation, but scaling to large and geometrically complex models remains future work.

Keywords: cutting simulation; position-based dynamics; surgery simulator; parallel computing; internal shape-preserving constraints

Check for updates

Academic Editor: Javid Taheri

Received: 30 June 2026
Revised: 1 August 2026
Accepted: 20 August 2026
Published: 24 August 2026

Copyright: © 2026 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license.

1. Introduction

Simulating cutting processes is a fundamental challenge in computer graphics and physically based modeling because it requires both geometric modifications and complex physical responses. Cutting simulation has numerous applications in animation, digital modeling, biomechanics, and medical training. With the rapid advancement of virtual reality (VR) and augmented reality (AR), simulation-based training has become increasingly important, particularly in medicine. Compared with traditional training methods, VR-based training can improve operation time, instrument and tissue handling, and reduce errors during laparoscopic operations [1]. There is also evidence that VR-based neuroanatomy education achieves learning outcomes comparable to conventional teaching methods while providing greater engagement, enjoyment and learner motivation [2]. Perhaps the greatest advantages of VR and AR simulators are the ability to provide safe, repeatable practice and enhance procedural understanding [3,4].

One of the most difficult aspects of these systems is the simulation of cutting operations. Cutting typically requires updates of mesh topology while simultaneously handling nonlinear deformation behavior [5]. Traditional approaches commonly rely on mesh-based representations, particularly tetrahedral meshes, which provide accurate volumetric modeling but can be computationally expensive [6]. These methods are often coupled with Finite Element Method (FEM), enabling highly realistic material deformation at the expense of additional computational complexity [7]. To overcome these limitations, alternative approaches based on meshless or hybrid representations have been proposed. These techniques simplify topology management while maintaining acceptable physical accuracy. For example, the study in [8] introduces a meshless cutting method with efficient updates of shape functions and crack propagation. A hybrid framework that combines a high-resolution visual model with an efficient low-resolution mechanical model is presented in [9]. More recent studies have further addressed computational challenges through parallel implementations on multi-core CPUs and GPUs [10,11], as well as the use of bounding volume hierarchies to accelerate collision detection and geometric queries [12].

In this work, we present a cutting simulation framework that combines a surface-based mesh representation with internal shape-preserving constraints (ISPCs) [13]. Deformation is simulated using Position-Based Dynamics (PBD), while ISPCs maintain the structural integrity of the object throughout the cutting process. Unlike our previous work, the primary objective of this study is to improve computational efficiency by parallelizing cutting operations on the GPU. Another major performance bottleneck is the frequent transfer of data between the CPU and the GPU, which introduces synchronization overhead and stalls execution. To address this issue, we redesign the system architecture to minimize data transfers in both directions, thereby improving overall runtime performance. The remainder of the paper is organized as follows: Section 2 demonstrates the methodology of our approach and key techniques. Section 3 provides experimental data. Section 4 discusses the findings, limitations and future work.

2. Materials and Methods

2.1. Computational Model

The proposed method employs PBD as the core simulation framework. In PBD, an object is represented as a set of discrete points connected by constraints that directly regulate their positions rather than having forces acting on them [14]. This formulation is particularly well suited for real-time applications due to its numerical stability, robustness, and computational efficiency [15]. A study in [16] applied PBD to simulate cholecystectomy in real time with haptic feedback. In [17] the authors showed that PBD can be efficiently used in interactive AR-based medical education. More recent work in [18] proposed a

graph-based PBD method with shape-matching constraints which allows for real-time, flexible, progressive and discontinuous cuts on soft tissue. The study in [19] applied PBD simulation in the training of surgical robots to achieve near-perfect real-world cutting. The constraints in PBD are solved to obtain a position correction Δp, so the constraint function can be defined as $C(p + \Delta p) = 0$. Then the constraint function can be approximated by gradient vector $\nabla_p C(p)$:

$$C(p + \Delta p) \approx C(p) + \nabla_p C(p) \cdot \Delta p = 0 \tag{1}$$

To ensure that the positional correction Δp is aligned with the gradient, a scalar multiplier λ is introduced such that:

$$\Delta p = \lambda \nabla_p C(p) \tag{2}$$

λ is solved as follows after substituting Equation (2) in Equation (1):

$$\lambda = -\frac{C(p)}{|\nabla_p C(p)|^2} \tag{3}$$

Then the displacement of each vertex is calculated as follows:

$$\Delta p_i = -\frac{w_i C(p)}{\sum_j w_j |\nabla_{p_j} C(p)|^2} \nabla_{p_i} C(p) \tag{4}$$

where w_i is the inverted mass of the vertex p_i.

The simulation incorporates two types of constraints: distance and bending. Distance constraints connect pairs of points and control the distance between them, preventing excessive stretching or compression. The distance constraint function is derived as $C_d(p_1, p_2) = |p_1 - p_2| - d$, where d is the rest distance between p_1 and p_2. Then the position correction for each vertex is obtained as follows:

$$\Delta p_1 = -\frac{w_1}{w_1 + w_2}(|p_1 - p_2| - d)\frac{p_1 - p_2}{|p_1 - p_2|} \tag{5}$$

$$\Delta p_2 = \frac{w_2}{w_1 + w_2}(|p_1 - p_2| - d)\frac{p_1 - p_2}{|p_1 - p_2|} \tag{6}$$

Bending constraints preserve the angle between adjacent surface elements, reducing overextension and flipping. A bending constraint is built from two adjacent triangles (p_1, p_2, p_3) and (p_1, p_2, p_4). The bending constraint function is then defined as:

$$\begin{aligned} C_b(p_1, p_2, p_3, p_4) &= \arccos(n_1 \cdot n_2) - \varphi_0 \\ &= \arccos\left(\frac{(p_2-p_1)\times(p_3-p_1)}{|(p_2-p_1)|\times|(p_3-p_1)|} \cdot \frac{(p_2-p_1)\times(p_4-p_1)}{|(p_2-p_1)|\times|(p_4-p_1)|}\right) - \varphi_0 \end{aligned} \tag{7}$$

where n_1 and n_2 are normal vectors of each triangle and φ_0 is the rest angle between them. The derivatives with respect to the triangle points are calculated as

$$\nabla_{p_4} C = -\frac{1}{\sqrt{1 - d^2}}\left(\left(\frac{\partial n_2}{\partial p_4}\right)^T n_1\right) \tag{8}$$

$$\nabla_{p_3} C = -\frac{1}{\sqrt{1 - d^2}}\left(\left(\frac{\partial n_1}{\partial p_3}\right)^T n_2\right) \tag{9}$$

$$\nabla_{p_2} C = -\frac{1}{\sqrt{1-d^2}}\left(\left(\frac{\partial n_1}{\partial p_2}\right)^T n_2 + \left(\frac{\partial n_2}{\partial p_2}\right)^T n_1\right) \tag{10}$$

$$\nabla_{p_1} C = -\nabla_{p_2} C - \nabla_{p_3} C - \nabla_{p_4} C \tag{11}$$

Then the gradient of normalized cross-product is computed for each vertex:

$$q_4 = \frac{p_2 \times n_1 + (n_2 \times p_2)d}{|p_2 \times p_4|} \tag{12}$$

$$q_3 = \frac{p_2 \times n_2 + (n_1 \times p_2)d}{|p_2 \times p_3|} \tag{13}$$

$$q_2 = -\frac{p_3 \times n_2 + (n_1 \times p_3)d}{|p_2 \times p_3|} - \frac{p_4 \times n_1 + (n_2 \times p_4)d}{|p_2 \times p_4|} \tag{14}$$

$$q_1 = -q_2 - q_3 - q_4 \tag{15}$$

And the final position correction is calculated as:

$$\Delta p_i = -\frac{w_i \sqrt{1-d^2}(\arccos(d) - \varphi_0)}{\sum_j w_j |q_j|^2} q_i \tag{16}$$

Such position manipulations allow for realistic deformations and shape restoration. In the present work, time integration is performed using the semi-implicit Euler method, while constraint projection is implemented using a Jacobi-style solver.

However, for objects with complex geometries, distance and bending constraints alone are insufficient to recover from large deformations. Therefore, the proposed framework incorporates the ISPC, a specialized form of distance constraint that connects points within the object's interior to enforce global structural consistency and preserve its overall shape. Mathematically, the ISPC is identical to the distance constraint. Collectively, these links form an internal structure that provides the surface mesh with effective volumetric support. Previous studies [13,20] have demonstrated that ISPCs effectively limit excessive deformation, improve recovery of the original shape, and consequently preserve the overall volume of the object. An ISPC is created by casting a bounded ray of fixed length from a vertex v_i along its inverted vertex normal—n_i. The ray is tested against all live triangles, excluding triangles incident to v_i and triangles whose outward normal opposes the ray direction, and the nearest intersected triangle is selected. The constraint endpoint v_j is chosen as the corner of that triangle closest to the hit point, and the constraint is initialized with rest length $l_0 = |v_i - v_j|$ measured at creation time.

2.2. GPU-Accelerated Processes

To improve the computational performance of the cutting simulation, the proposed framework employs compute shaders available in Unity [21]. Compute shaders are general-purpose GPU programs written in High-Level Shading Language (HLSL) that enable highly parallel computation. By offloading computationally intensive tasks from the CPU to the GPU, thousands of threads can execute the same operation simultaneously. Unlike vertex and fragment shaders, compute shaders are independent of the graphics rendering pipeline and can therefore be used for arbitrary parallel computations. Their functionality is organized into kernels, which are executed according to predefined thread-group sizes.

In the proposed implementation, PBD constraint solving, triangle–triangle intersection detection, ISPC updates, and collision detection and response are all parallelized using compute shaders. During constraint solving, positional corrections are accumulated atomically in per-vertex buffers to prevent race conditions. Compared with our previous work,

the computationally expensive triangle–triangle intersection test is offloaded to the GPU, allowing all triangle pairs to be processed concurrently instead of performing sequential intersection tests on the CPU. The detected intersections are accumulated in an IntersectBuf compute buffer and subsequently transferred to the CPU for further processing. The remaining stages of the cutting algorithm are executed on the CPU because they rely on mesh adjacency information, which requires dynamic dictionary-like data structures that are not efficiently supported on the GPU. After the cutting operation is completed, the newly generated vertices and triangles are uploaded to the GPU compute buffers. All geometry buffers are allocated once at initialization at a fixed multiple of the loaded mesh size. To further reduce data-transfer overhead, surface rendering is performed directly from these compute buffers, allowing both simulation and rendering to operate on the same geometry data. This architecture minimizes synchronization and memory transfers between the CPU and the GPU, resulting in improved overall simulation performance.

2.3. Collision Handling

A vertex-triangle self-collision is implemented in the proposed method. To accelerate the collision test, a spatial hash [22] over triangles is used and rebuilt in every frame. Each triangle gathers candidates from its $3 \times 3 \times 3$ cell neighborhood, and the vertex-triangle test detects crossings. Topological neighbors, i.e., triangles that share a vertex, are skipped from the collision test. Positional corrections are also accumulated atomically. Vertices created during an active cut are excluded until the cut finishes and settles, because their coincident seam duplicates would otherwise explode.

2.4. Cutting Simulation

The cutting procedure in the proposed method is consistent with the four main steps described in the previous study [20], which include detecting intersections between the cutting tool and the object, subdividing affected triangles, filling gaps in the mesh and updating simulation constraints. The summary of the cutting algorithm is presented in Algorithm 1.

To detect collisions between the cutting tool and surface mesh, the blade is discretized as a line segment defined by two endpoints. A sweeping plane is dynamically constructed from the positions of these endpoints in the current and previous frames, representing the blade's motion over time. The sweeping plane is then decomposed into two triangles, which are tested against the mesh triangles using the signed tetrahedron volume test to identify intersections. In the current implementation, a point exactly on the sweeping plane, as well as segments passing through a swept-quad edge or vertex, fails the crossing test. Therefore, coplanar edges and touching-without-crossing instances all report no intersection. In cases where a cut passes near a vertex, the intersection point is snapped to the vertex in order to avoid too small and sliver triangles. During the intersection test, information such as the number of intersected edges and vertices is recorded for each affected triangle. To improve computational performance, all triangle–triangle intersection tests are performed in parallel on the GPU using Unity compute shaders.

After the triangle–triangle intersection test is completed, the detected intersection data are transferred to the CPU, where a four-state machine is updated according to the intersection results. The four cutting states include "cut start", "cutting", "cut end" and "no cutting". The state machine is updated depending on whether the front and back edges of the sweeping plane were intersected or not. The front edge is the line segment constructed from the current positions of cutting tool's endpoints. The back edge is the line segment between previous frame positions of the cutting tool's endpoints. "Cut start" is detected once the front edge is intersected with any surface triangle. "Cutting" state is enabled when

both front and back edges are intersected. "Cut end" is set when there is no intersection of the front edge, but the back edge is still crossed. "No cutting" state is determined when none of the edges is intersected. State transitions determine whether the current step performs cutting operations or finalizes the cut by rebuilding the constraints and resetting data. Then intersection information is used to define the intersection configuration of each affected triangle. Figure 1 illustrates the three intersection configurations implemented in the proposed method. In Figure 1a a triangle with two intersected edges is subdivided into three new triangles. In Figure 1b a triangle with one intersected edge and one intersected vertex is divided into two new triangles. And Figure 1c shows the case in which two intersected vertices require duplication of the shared edge to separate the adjacent triangles along the cutting path. Each operation records the new open boundary edges on both sides of the cut, updates the adjacency information, and queues constraints for removal and creation.

Algorithm 1. Progressive cutting (executed once per frame)

Input: Cutting tool endpoints—current p0, p1, previous p0′, p1′
Geometry data—vertices list V, triangles list T, adjacency
Constraints—distance, bending, ISPC
Cutting state s = {NO-CUT, CUT_START, CUTTING, CUT_END}
Output: updated V, T, adjacency, distance, bending, ISPC, s
1: [CPU] Build sweeping quad from {p0, p1, p0′, p1′}; split into two triangles Q
2: [GPU] parallel for each triangle t ∈ T:
3: test t against Q via signed-tetrahedron volume
4: record intersected edge/vertex counts and hit points into *IntersectBuf*
5: [SYNC] read back intersection summary from *IntersectBuf*: GPU to CPU transfer
6: [CPU] update state machine s from summary
7: if s ∈ {CUT_START, CUTTING} do cutting
8: [CPU] for each intersected triangle:
9: classify configuration (Figure 1)
10: subdivide; append new V, T to pre-allocated buffers via atomic counters
11: record open boundary edges per cut side; update adjacency
12: queue stale constraints for removal, edges for adding new constraints
13: if s = CUT_END: finalize
14: [CPU] fill seam holes: ear-clipping triangulation + Delaunay refinement
15: append patch V, T to buffers via counters
16: [CPU] remove queued constraints
17: [CPU] add distance/bending constraints for new triangles
18: [CPU] add ISPC: locate internal pairs via spatial-grid traversal
19: [SYNC] push appended V/T/constraint to GPU buffers; update counts
20: [GPU] surface rendering reads geometry buffers directly

Because the proposed method operates on a surface mesh, cutting introduces openings in the geometry that must be closed to preserve a watertight surface. These openings are filled using an ear-clipping triangulation algorithm [23], which reuses the existing boundary vertices. To maintain mesh quality and preserve the original resolution, the newly generated triangles are further refined using a Delaunay-style subdivision approach [24].

Each cutting operation modifies the mesh geometry and therefore requires updates to the corresponding GPU data structures. To avoid reallocating and reinitializing the

compute buffers after every cut, all geometry buffers are pre-allocated with additional capacity and maintain element counters that track the current number of valid vertices, triangles, and constraints. Newly generated elements are appended to the corresponding buffers, and the associated counters are updated accordingly. Furthermore, to minimize costly readbacks from the GPU, geometry data are maintained on both the CPU and the GPU, allowing only the modified elements to be synchronized after each cutting operation.

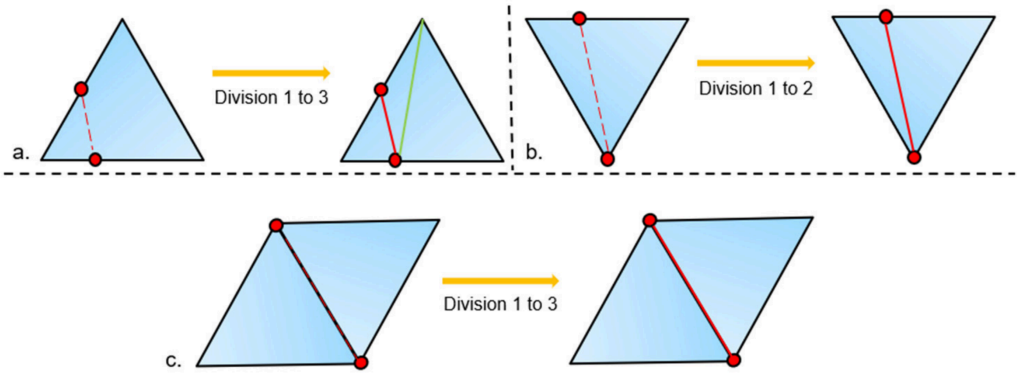

Figure 1. Triangle subdivision schemes: (**a**) Subdivision 1 to 3. (**b**) Subdivision 1 to 2. (**c**) Edge duplication. Solid black lines show the original triangle edges; dotted red lines mark where the cutting tool intersects a triangle; solid red and green lines are the new edges created by subdivision.

Once the geometry has been updated, the PBD constraints are reconstructed to reflect the modified mesh topology. Constraints associated with removed elements are discarded, while new constraints are generated for the newly created vertices and triangles. Similar to the geometry buffers, the constraint buffers are overallocated to accommodate newly appended constraints without requiring complete buffer reinitialization. During cutting the ISPCs are updated once per cut step. A live ISPC is removed if either its segment intersects the sweeping plane or its endpoints belong to different cut parts. The first criterion severs constraints during an in-progress cut, while the second one guarantees retirement of any residual cross-part bridge once the mesh separates. Creation is then re-run for every vertex not currently owning a live constraint, using the same inward ray cast. A spatial grid combined with ray marching is employed to efficiently identify candidate connections, eliminating the need for an expensive search over all mesh triangles. Candidate triangles in different cut parts than the source vertex are rejected, so that no constraint may ever bridge separated cut parts.

3. Results

The proposed method was implemented in Unity version 2022.3.30f1 using C# v9.0 and tested on a system equipped with an Intel i7-10700KF CPU 3.80 GHz (Intel Corporation, Santa Clara, CA, USA), an NVIDIA GeForce RTX 3060 Ti GPU (NVIDIA Corporation, Santa Clara, CA, USA) with 32 GB of RAM and 8 GB of V-RAM. The information about the models used in cutting experiments is presented in Table 1. All experiments were conducted with a 0.005 timestep and 30 iterations per frame. Stiffness of distance and bending constraints was set to 0.5, and the corresponding value for the ISPC was 0.3. It must be noted that in case of the Octopus model, the stiffness of all three types of constraints was set to 1 to increase the volume preservation. Epsilon values in the range 10^{-7}–10^{-12} were used.

Table 1. Information about 3D models used in the cutting experiments.

	Sphere 20	Sphere 80	Torus	Bunny	Octopus	Banana	Liver	Armadillo
Vertices	12	42	400	752	1410	2798	3183	15,002
Triangles	20	80	800	1500	2816	5592	6362	30,000
Distance Constraints	30	120	1200	2250	4224	8388	9543	45,000
Bending Constraints	30	120	1200	2250	4224	8388	9543	45,000
ISPC	6	21	200	537	958	2262	2354	10,710

3.1. Progressive Cutting Experiments

Cutting experiments were conducted on a set of static models. Figures 2 and 3 illustrate the cutting process and subsequent deformation of the Sphere 20, Sphere 80, Torus, Bunny, Octopus, Banana, Liver and Armadillo models. The results demonstrate that the proposed subdivision strategy produces precise cutting surfaces that closely follow the trajectory of the cutting tool. After the cutting operation, gravity and collision handling were enabled, allowing the separated parts to interact naturally with each other and with the ground. Despite undergoing substantial deformation during impact, all models retained their structural integrity owing to the presence of ISPCs. Higher-resolution meshes experienced larger deformations, but they successfully recovered their original shape while maintaining their overall volume after the collision. Each model was cut 10 times, and quantitative performance data were obtained.

Table 2 presents mean frame rate measured in FPS and deformation, collision and cutting times in ms. Each model was profiled over a 10 s window at 0.1 s intervals. However, it must be noted that in Unity, because dispatches are submitted asynchronously, a host-side timer placed around a stage measures command submission rather than execution. To obtain per-stage execution times we insert an explicit synchronization point at each stage boundary, a blocking readback of a single-element buffer, which forces the command queue to drain before the host-side timer is stopped. Each interval therefore reports wall-clock time from the start of the stage until its final dispatch has retired on the device. This instrumentation serializes host and device. In normal operation the host submits substep n + 1, while the device executes substep n. Under instrumentation, this overlap is removed, and each stage begins with an empty pipeline. The per-stage times reported here are consequently an upper bound on each stage's contribution to an uninstrumented frame, and the per-stage figures do not sum to the measured frame time. We therefore report the two quantities from separate runs under identical scene, model and parameter settings. Frame times are measured without instrumentation as the interval between successive frame starts, with vertical synchronization disabled. Stage breakdowns are measured in a separate instrumented run. Cutting time is reported only for those frames in which a topology update actually occurred, so it is a conditional mean, not a per-frame mean.

Table 3 reports standard deviation (SD), median, P99 and P95 values of per-stage time. It is noticeable that in all models, except for Armadillo, deformation and collision time dominates over cutting time. From Torus upward, the split between deformation and collision is remarkably stable with a little greater impact of deformation time across five models. The two trivial sphere models are the exception, where deformation's fixed iteration overhead dominates a collision workload that is essentially empty. The mid-size models (Bunny, Octopus, Banana, and Liver) are highly stable, with P99 values within 25% of the median in the deformation and collision stages. Their distributions are close to symmetric, indicating steady-state behavior with no significant stalls. Armadillo shows genuine structural dispersion: the cutting SD is 18.949 ms against a conditional mean of 11.123 ms, with a maximum of 72.419 ms, which contributes the most compared with other stages. Its median cutting time is only 0.047 ms, so the distribution is extremely

heavy-tailed: the majority of cut frames are cheap, and a small minority incur very large retriangulation costs as well as constraint update. Against a 30 Hz interactive budget, all models clear it on the mean except Armadillo. Figures 4 and 5 demonstrate average FPS rate evolution over the 10 s time period in all eight models. A detailed representation of total simulation, deformation, collision and cutting time of each model can be found in Appendix A, Figure A1.

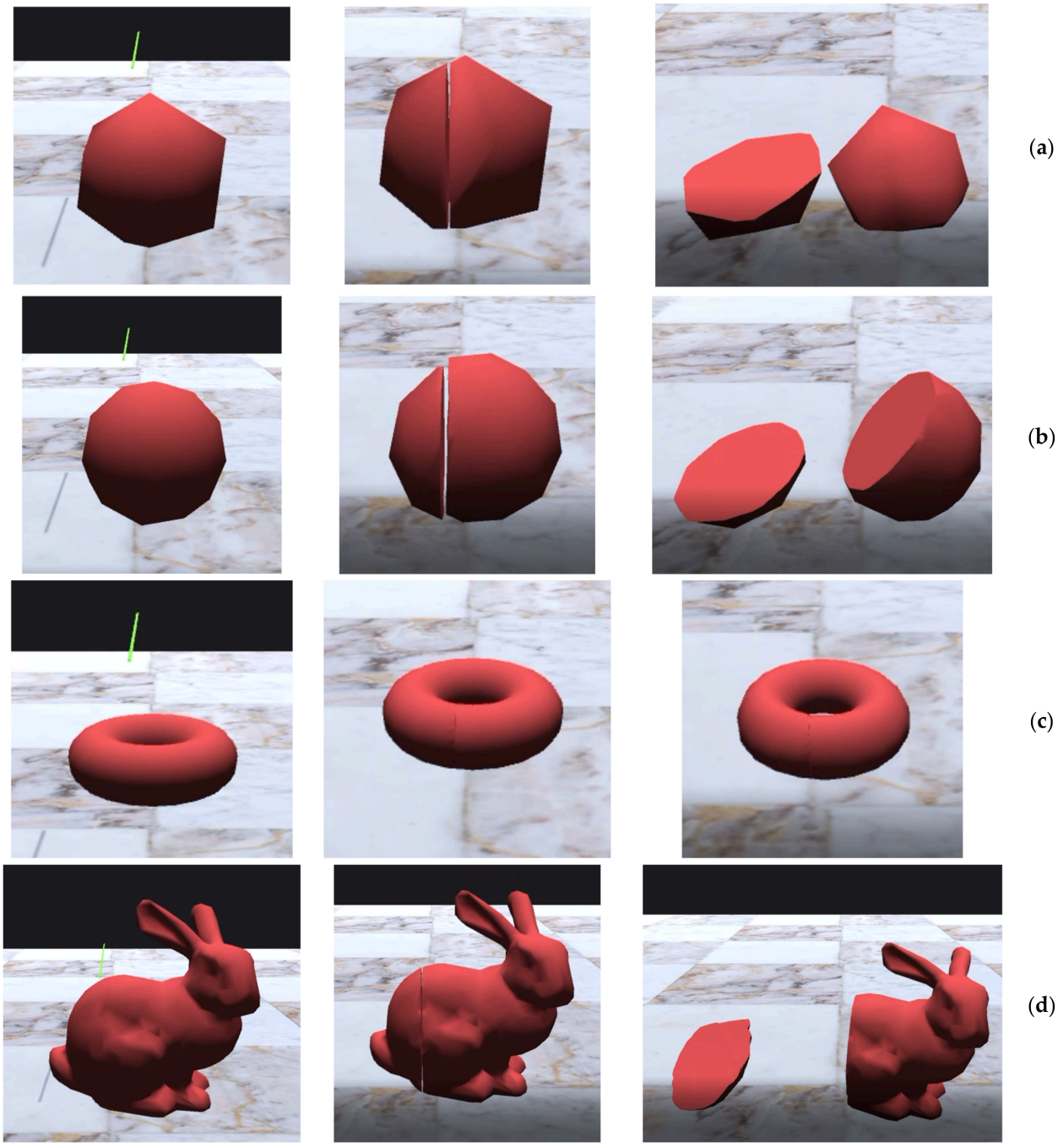

Figure 2. Cutting simulation results: (**a**) Sphere 20; (**b**) Sphere 80; (**c**) Torus; (**d**) Bunny. The green line in the figures represents the cutting tool trajectory.

CPU and GPU time queries were evaluated. Per-frame timings were logged at approximately 0.1 s intervals, aligned across runs on a common 0.1 s grid, and averaged bin-wise over a 10 s steady-state window positioned after the start-up transient. Frames in which no cutting occurred were excluded from the cutting statistics. Figure 6 demon-

strates per-stage time series of average CPU and GPU time and histogram analysis of each model. Deformation and self-collision together accounted for over 95% of frame time in all cases except for Armadillo. Cutting contributed negligibly in the median but exhibited a strongly right-skewed distribution, with per-frame maxima increasing from 0.283 ms for Sphere 20 to 4.976 ms for Banana and 7.640 ms for Liver. These peaks coincided with cut-finalization frames, in which seam triangulation and constraint rebuilding are performed on the CPU. Torus was the sole exception to the observed stability: its cost approximately tripled, from roughly 5.6 ms to 18 ms, once the cut completed and the separated parts entered sustained contact.

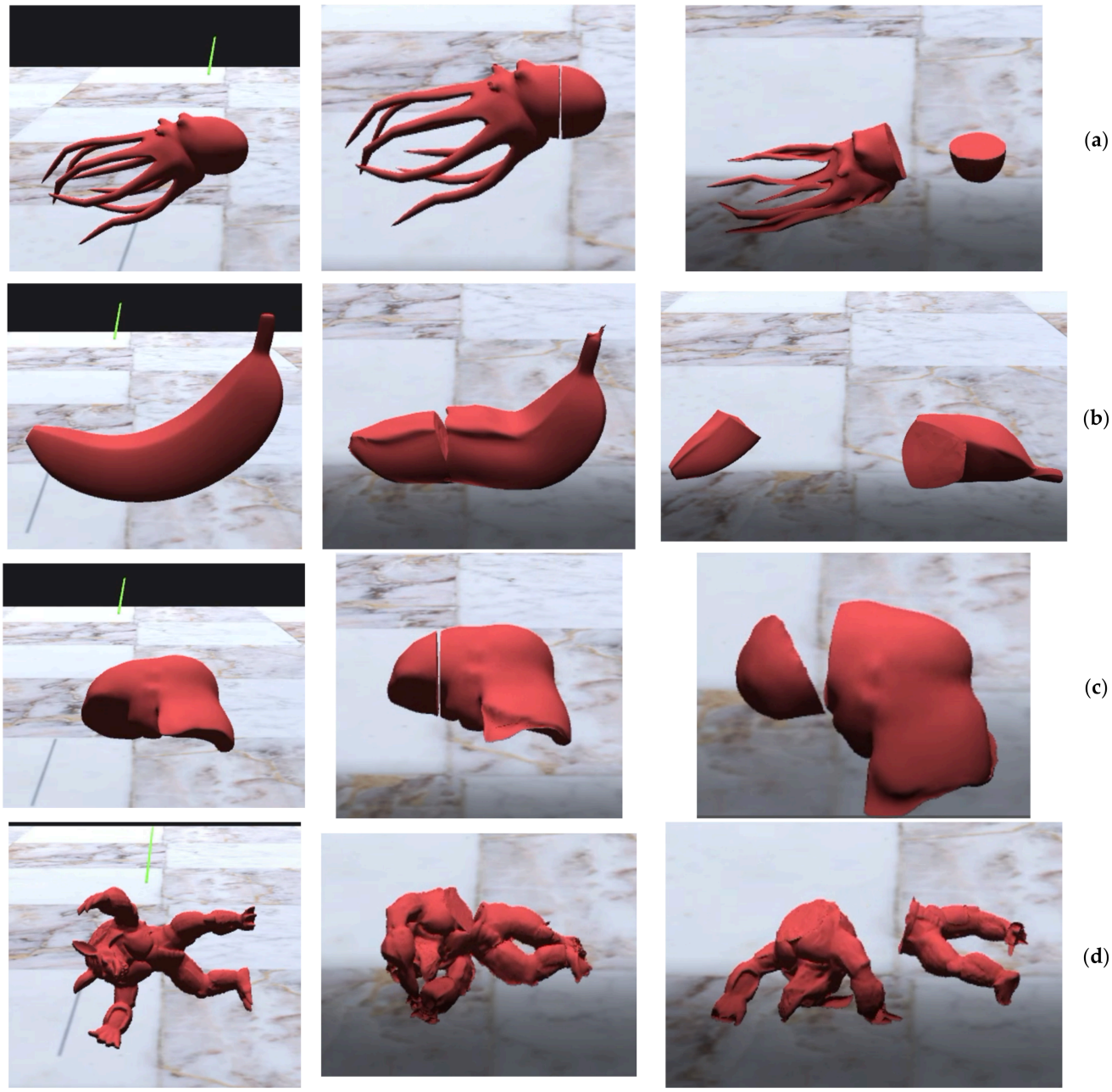

Figure 3. Cutting simulation results: (**a**) Octopus; (**b**) Banana; (**c**) Liver; (**d**) Armadillo. The green line in the figures represents the cutting tool trajectory.

Table 2. Mean per-frame timing and per-stage timings with GPU ISPC method.

Model	Frame Rate in FPS, Mean (min, max)	Deformation Time in ms, Mean (min, max)	Collision Time in ms, Mean (min, max)	Cutting Time in ms, Mean (min, max)
Sphere 20	481.04 (323.71, 629.86)	0.854 (0.684, 3.901)	0.209 (0.130, 0.938)	0.034 (0.000, 0.283)
Sphere 80	487.47 (327.96, 609.55)	1.004 (0.764, 1.436)	0.348 (0.239, 0.696)	0.023 (0.000, 0.368)
Torus	473.90 (280.85, 602.22)	5.835 (2.871, 13.074)	5.089 (1.807, 9.688)	0.062 (0.0001, 0.857)
Bunny	449.16 (174.19, 591.39)	11.131 (10.273, 11.821)	9.862 (8.064, 11.960)	0.894 (0.0320, 4.791)
Octopus	401.94 (94.74, 554.11)	12.564 (11.946, 13.040)	11.409 (8.759, 14.415)	0.251 (0.0001, 2.661)
Banana	221.66 (27.17, 325.21)	12.984 (12.50, 15.676)	12.375 (11.880, 15.674)	0.914 (0.0017, 4.976)
Liver	147.01 (15.70, 241.02)	12.825 (12.155, 13.350)	10.487 (9.050, 12.928)	1.319 (0.0004, 7.640)
Armadillo	5.12 (0.79, 26.11)	15.262 (14.018, 32.162)	12.655 (9.956, 25.375)	11.123 (0.0002, 72.419)

Table 3. Standard deviation, median, P95 and P99 values of deformation, collision and cutting times for each model with GPU ISPC method.

Model	Deformation Time				Collision Time				Cutting Time			
	SD	Median	P95	P99	SD	Median	P95	P99	SD	Median	P95	P99
Sphere 20	0.327	0.811	1.066	1.583	0.086	0.116	0.268	0.384	0.064	0.051	0.165	0.250
Sphere 80	0.151	1.389	1.739	1.984	0.083	0.337	0.490	0.641	0.066	0.003	0.081	0.299
Torus	2.703	4.885	9.053	9.297	3.267	3.061	9.342	9.553	0.172	0.009	0.315	0.780
Bunny	0.530	11.38	11.721	11.785	1.042	9.554	11.631	11.915	1.205	0.351	3.462	4.455
Octopus	0.356	12.735	13.007	13.030	2.118	9.749	14.354	14.412	0.583	0.026	1.109	2.593
Banana	0.392	13.090	13.358	13.582	0.774	12.087	14.477	15.615	1.456	0.054	4.094	4.801
Liver	0.435	13.067	13.286	13.335	1.533	9.270	12.869	12.925	2.303	0.024	6.636	7.469
Armadillo	2.294	14.609	17.664	24.805	3.375	10.360	18.620	19.665	18.949	0.047	57.943	71.28

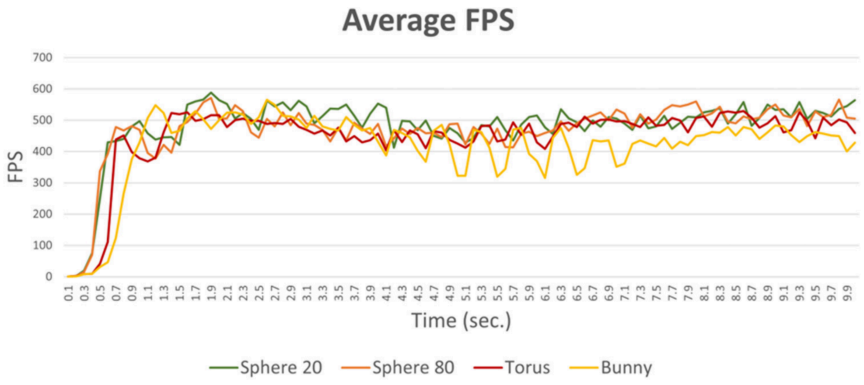

Figure 4. Average FPS of Sphere 20, Sphere 80, Torus and Bunny.

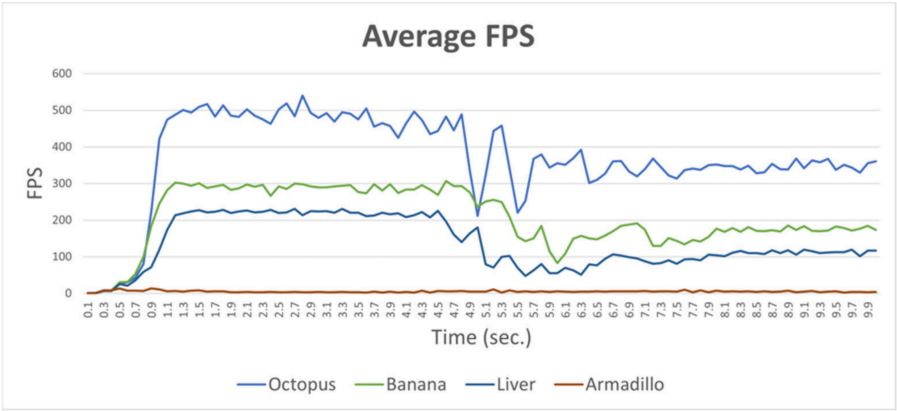

Figure 5. Average FPS of Octopus, Banana, Liver and Armadillo.

The volume preservation was evaluated for all eight test models: Sphere 20, Sphere 80, Torus, Bunny, Octopus, Banana, Liver, and Armadillo. The initial volume was calculated using the divergence theorem [25] on surface triangles. During simulation, volume was logged as a percentage of the deformable body's initial volume. Table 4 summarizes the volume-preservation statistics for all eight models. Figure 7 shows the corresponding averaged volume curves. All eight models exhibit a common three-phase response. Volume is preserved exactly at 100% during an initial quiescent phase; a rapid decrease follows, initiated between 3.5 s and 6.9 s after the start of the window and coinciding with the completion of the cut and the subsequent impact of the separated parts with the ground plane; and a stable plateau is then established and persists to the end of the observation period. The models separate into two clearly distinct groups with respect to the magnitude of this response, with no intermediate cases observed. The first group—Sphere 20, Sphere 80, Torus and Bunny—retains essentially its full volume, with averaged minima not below 96.37%. Sphere 20 is the most stable model overall (mean 99.97%, SD 0.04%). These results indicate that ISPCs restore the original volume effectively for low- and moderate-resolution meshes of simple topology and that a non-trivial topology such as the torus does not by itself impair volume preservation.

Figure 6. *Cont.*

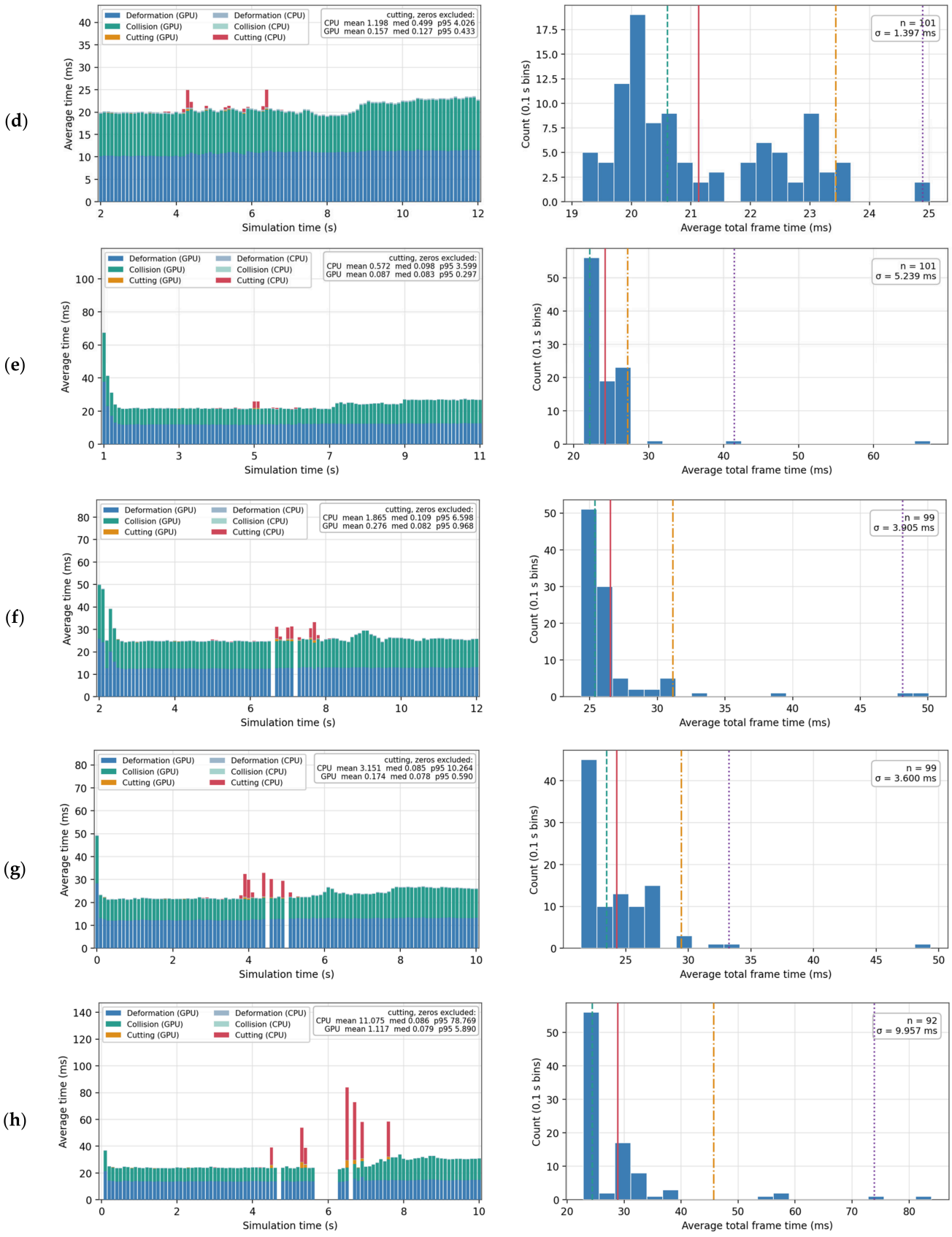

Figure 6. Per-stage time series of average CPU and GPU time in ms on the left and histogram analysis on the right: (**a**) Sphere 20; (**b**) Sphere 80; (**c**) Torus; (**d**) Bunny; (**e**) Octopus; (**f**) Banana; (**g**) Liver; (**h**) Armadillo. The solid red line indicates the mean, the dashed green line the median, the dash-dotted yellow line the 95th per-centile (P95), and the dotted purple line the 99th percentile (P99).

The second group—Liver, Banana, Armadillo and Octopus—exhibits substantially larger deficits. Liver, Banana and Armadillo behave comparably to one another, settling at plateaus of 92.0–93.3% after minima of 74.57–87.43%. For these three models the median volume remains at 100%, indicating that the flat pre-collision phase occupies more than half of the observation window and that the deficit is confined to the post-collision interval. The Octopus model shows markedly poorer volume preservation than any other model tested. Its decline begins the earliest, is the steepest observed—a decrease of approximately 27%—and is followed by no recovery whatsoever. The averaged volume stabilizes at 73.43%. Taken together, these results confirm that the proposed ISPC formulation maintains volume to within 0.4% for models of low geometric complexity but that its effectiveness degrades as mesh complexity increases, with the highly detailed Octopus model representing the limiting case. A detailed histogram of the per-frame sampled volume percentage of each model can be found in Appendix A, Figure A2.

Table 4. Volume-preservation statistics over 10 s observation window with GPU ISPC method. All values in percent of initial volume.

Model	Mean	Median	P95	P99	SD	Min
Sphere 20	99.967	99.956	100	100	0.042	99.78
Sphere 80	99.858	99.816	100	100	0.137	99.55
Torus	99.805	99.782	100	100	0.409	96.37
Bunny	99.791	99.825	100	100	0.239	98.94
Octopus	85.259	73.575	100	100	13.171	72.97
Banana	96.853	100	100	100	4.853	74.57
Liver	97.550	100	100	100	3.533	87.43
Armadillo	97.928	100	100	100	4.079	86.73

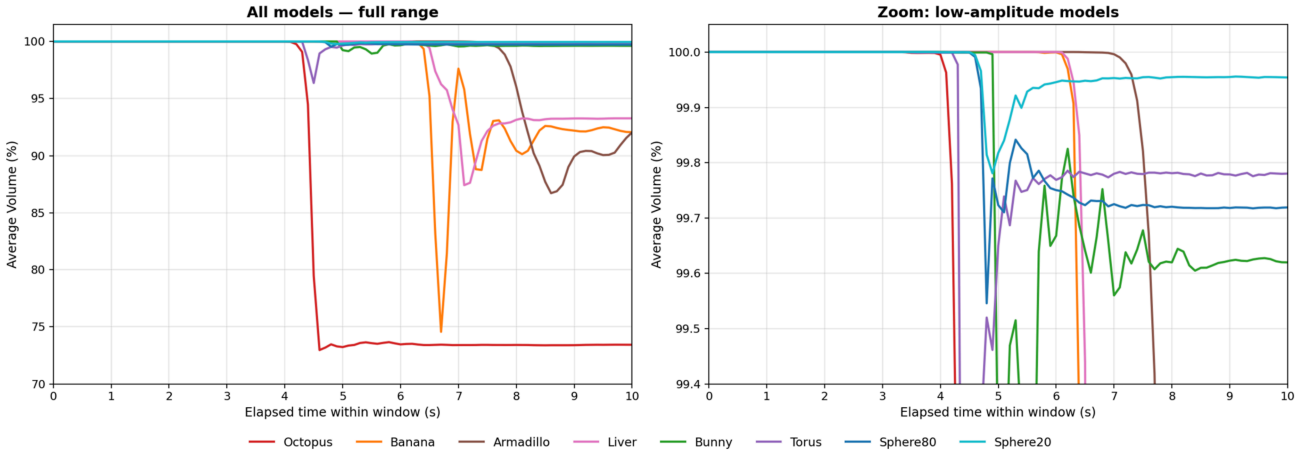

Figure 7. Average volume of Sphere 20, Sphere 80, Torus, Bunny, Octopus, Banana, Liver and Armadillo.

3.2. Multiple-Cutting Experiment

Multiple-cutting experiments on the Sphere 80, Torus, and Banana models are presented in Figures 8a, 8b and 8c, respectively. During these experiments, the user was able to move the cutting tool freely, allowing multiple cuts to be performed on each model. After the cutting operations were completed, gravity was enabled, causing the separated parts to fall onto the ground. As shown in Figure 8, the detached parts interacted naturally with the ground and with one another, exhibiting realistic deformation and collision response throughout the simulation.

To quantify computational performance during free multiple cutting, five independent runs were recorded for each of the Sphere 80, Torus, and Banana models, and

per-frame stage timings were sampled every 0.1 s. Analysis windows of 1.5–13.5 s (Sphere 80), 2–17 s (Torus), and 3–17 s (Banana) were selected to exclude engine warm-up transients while covering the complete cutting and post-cut interaction phases, yielding 121, 151, and 141 averaged samples, respectively. Figure 9 shows per-stage time series of average CPU and GPU time in ms and histogram analysis of each model. The Sphere 80 model maintained real-time performance throughout the experiment, with a mean total frame time of 1.81 ms (median 1.42 ms, SD 0.77 ms) and tail latency of 3.14 ms for P95 and 3.90 ms for P99. The Torus model averaged 4.14 ms (median 2.35 ms, SD 3.85 ms, P95 13.69 ms, and P99 14.15 ms) and the Banana model 14.37 ms (median 10.31 ms, SD 7.28 ms, P95 26.52 ms, and P99 27.53 ms). For the two larger models, the mean substantially exceeds the median because the frame-time distributions are bimodal: a stable pre-cut regime is followed by a distinct post-cut regime that begins once the separated parts fall under gravity and the collision response is activated.

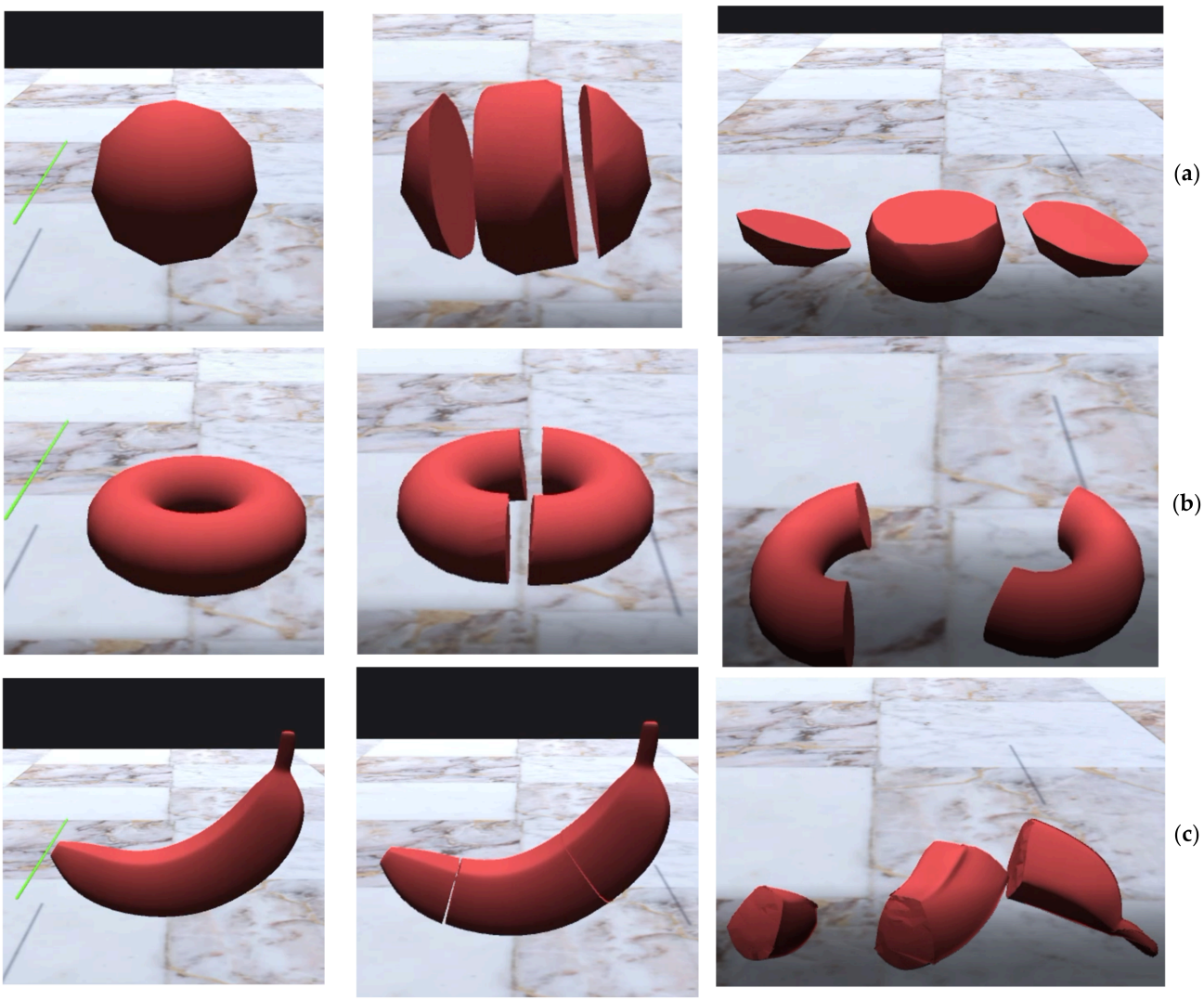

Figure 8. Multiple cuts are performed on: (**a**) Sphere 80; (**b**) Torus; (**c**) Banana. The green line in the figures represents the cutting tool trajectory.

The stage breakdown shows that deformation dominated the total cost in all three models, averaging 1.57 ms for Sphere 80, 3.18 ms for Torus, and 10.64 ms for Banana. Collision detection was inactive during the pre-cut phase and became the second-largest contributor after separation, averaging 0.67 ms, 4.32 ms, and 9.06 ms in collision-active frames for the

three models. In the Torus and Banana models, post-cut collision cost approached the same order as deformation, which explains the transition to the second frame-time regime. In contrast, the cutting stage itself remained a minor contributor: in frames where cutting work was performed, it averaged only 0.023 ms for Sphere 80, 0.028 ms for Torus, and 0.456 ms for Banana, corresponding to approximately 1.3% of total accumulated frame time for the largest model. These results indicate that the GPU-based cutting pipeline introduces negligible per-frame overhead even under repeated free cutting and that overall performance after cutting is governed primarily by deformation and inter-part collision handling rather than by topology modification.

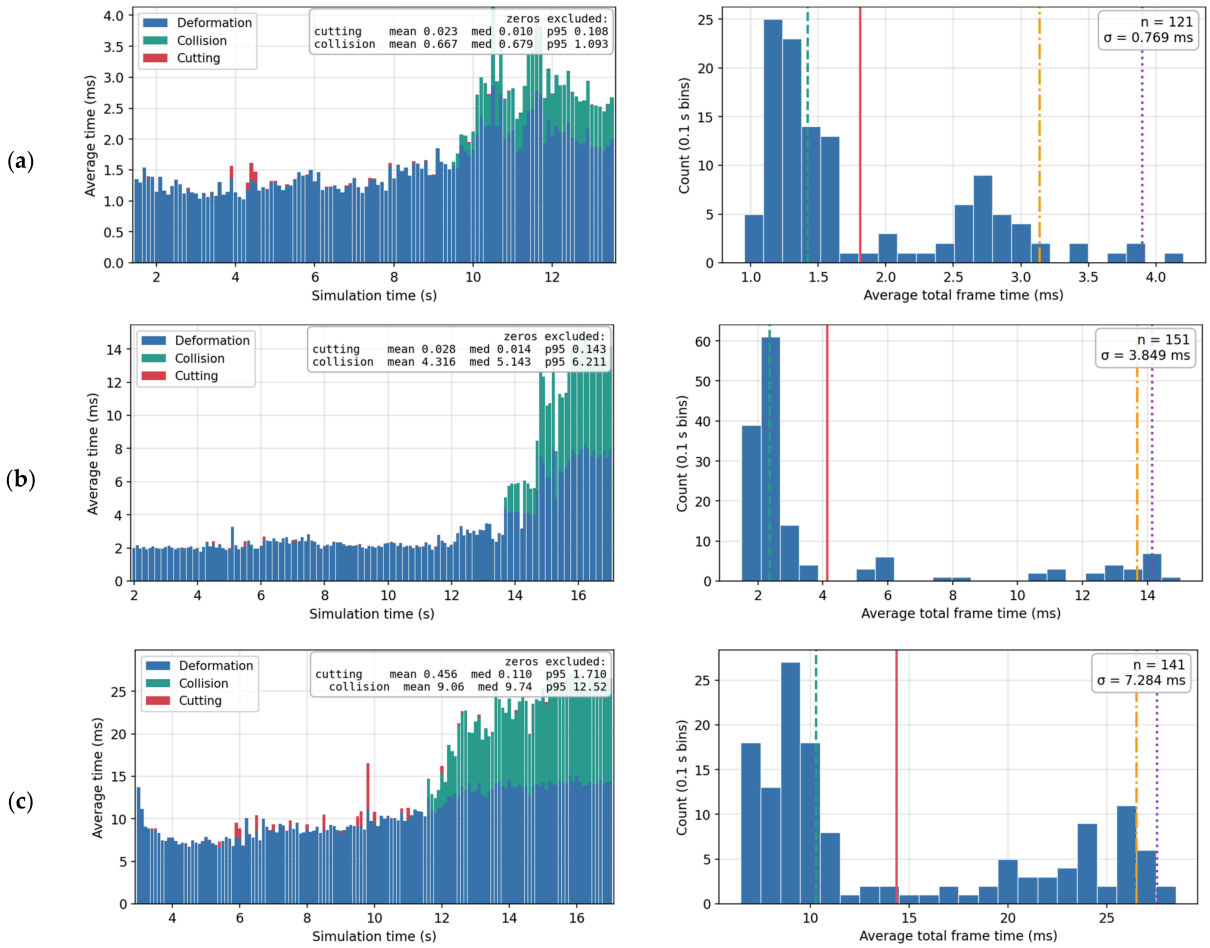

Figure 9. Per-stage time series of average CPU and GPU time in ms on the left and histogram analysis on the right: (**a**) Sphere 80; (**b**) Torus; (**c**) Banana. The solid red line indicates the mean, the dashed green line the median, the dash-dotted yellow line the 95th percentile (P95), and the dotted purple line the 99th percentile (P99).

To evaluate mesh growth during multiple cutting, the numbers of newly generated vertices and triangles were recorded for the first and second cuts across five independent test runs on the Sphere 80, Torus, and Banana models. For the Banana model, the first cut produced on average 696 new vertices (SD 88) and 1493 new triangles (SD 176), corresponding to 24.9% and 26.7% of the original mesh (2798 vertices and 5592 triangles). The second cut generated 853 vertices (SD 83) and 1807 triangles (SD 166), or 30.5% and 32.3% of the original counts, so that after two cuts, the mesh had grown by 55.4% in vertices and by 59.0% in triangles. The Torus model exhibited comparable relative growth: the first cut added 112 vertices and 262 triangles (28.0% and 32.8% of the original 400 vertices and 800 triangles) and the second cut 119 vertices and 276 triangles (29.8% and 34.5%), for

cumulative increases of 57.8% and 67.2%. For the smallest model, Sphere 80, each cut generated more new elements than the entire original mesh contained—52 vertices and 114 triangles for the first cut (123.8% and 142.5% of the original 42 vertices and 80 triangles) and 53 vertices and 122 triangles for the second one (126.2% and 152.8%)—so that two cuts approximately tripled the mesh size (+250.5% vertices and +294.8% triangles). This behavior reflects the fact that the number of intersected triangles along a cutting path cannot fall below a geometric minimum, making the relative topological cost of cutting inversely related to mesh resolution. The ratio of new triangles to new vertices remained stable at approximately 2.1–2.3 across all models and both cuts, consistent with the subdivision schemes in Figure 1, in which each duplicated cut-path vertex is shared by roughly two newly created triangles per side. Despite this substantial mesh growth, the cutting-stage frame cost remained below 0.5 ms on average for all models.

3.3. Comparison with Previous Works

A computational performance comparison was conducted between the proposed GPU-based ISPC method, our previously published slicing method [26] and the CPU-based ISPC method [20]. Table 5 describes the algorithm differences in the three compared methods. For better comparison, the cutting tool moved along a predefined trajectory to closely follow the cutting path in the previous studies. Table 6 summarizes the average frame rate (FPS) achieved by all three approaches. The last two columns give the FPS improvement (%) of the proposed GPU ISPC method over the Slicing and CPU ISPC methods. Compared with the slicing method, the proposed GPU-based ISPC approach achieved FPS improvements of 133%, 175%, 462%, 677%, and 590% for the Sphere 20, Sphere 80, Bunny, Octopus, and Banana models, respectively. The only exception was the Armadillo model, for which the slicing method achieved a higher frame rate, 1.53% greater. A comparison with the CPU-based ISPC implementation demonstrated performance improvements of 179%, 193%, 395%, and 327% for the Sphere 20, Sphere 80, Octopus, and Banana models, respectively. The Bunny and Armadillo models were excluded from this comparison because they were not evaluated in [20], while the Torus model was omitted because it was not included in either of the previous studies.

Table 5. Algorithm changes in the three compared methods.

	Slicing Method [26]	**CPU ISPC Method [20]**	**GPU ISPC Method**
Deformation method	GPU-based PBD with distance and bending constraints.	GPU-based PBD with distance, bending, and internal shape-preserving constraints.	GPU-based PBD with distance, bending, and internal shape-preserving constraints.
Volume preservation	No volume preservation.	Volume is preserved due to implementation of ISPCs. ISPCs are regenerated after cutting.	Volume is preserved due to implementation of ISPCs. Exhaustive search for ISPC regeneration is replaced with spatial-grid traversal and ray marching to find internal connection candidates.
Collision handling between cut parts	None.	GPU-based collision handling uses AABBs in the broad phase and the Moller–Trumbore algorithm in the narrow phase. A velocity-based collision response is applied.	A vertex-triangle collision using a spatial hash is implemented in GPUs. Each triangle gathers candidates from its $3 \times 3 \times 3$ cell neighborhood, while topological neighbors are skipped.

Table 5. *Cont.*

	Slicing Method [26]	CPU ISPC Method [20]	GPU ISPC Method
Cutting	Cutting is processed with a static plane. The intersected triangles are removed, then the boundary is reconstructed, the cut parts are patched with new triangles and refined. Only complete single slices are supported. Algorithm is implemented purely in CPUs.	Progressive cutting with a moving cutting tool is implemented. Triangles are subdivided along the exact intersection path. Seam vertices are duplicated to separate the cut parts. The algorithm is implemented purely in CPUs.	Cutting process is similar to CPU ISPC method; however the triangle–triangle intersection test is performed on GPUs. A four-state cutting state machine is implemented ("no cut", "cut start", "cutting", and "cut end"). Cutting operations run during "cut start" and "cutting", while hole filling, constraint rebuilding and buffer synchronization are deferred to "cut end".
Intersection detection	Brute force: each surface triangle is tested against two sweep-plane triangles using triangle-edge test.	Oriented Bounding Box is used in the broad phase, while BFS traversal over precomputed triangle adjacency is used in the narrow phase.	Parallelized brute force triangle-edge test on the GPU. The intersection results are accumulated in the specified buffer and read back to CPU.
Rendering method	Unity's internal Mesh Renderer is used. Requires readback from the GPU of the updated vertex positions in each frame.	Unity's internal Mesh Renderer is used. Requires readback from the GPU of the updated vertex positions in each frame.	The models are rendered directly from GPU geometry buffers using Unity's DrawProcedural() function. Therefore, there is no need for data transfer between the GPU and the CPU.

Table 6. Performance evaluation of CPU ISPC and GPU ISPC methods.

	Slicing Method [26], avr. FPS	CPU ISPC Method [20], avr. FPS	GPU ISPC Method, avr. FPS	GPU ISPC-to-Slicing FPS Increase (%)	GPU ISPC-to-CPU ISPC FPS Increase (%)
Sphere 20	205.9	172.1	481.04	133	179
Sphere 80	177.2	166.1	487.47	175	193
Bunny	79.9	-	449.16	462	-
Octopus	51.7	81.1	401.94	677	395
Banana	32.1	51.8	221.66	590	327
Armadillo	5.2	-	5.12	−1.53	-

4. Discussion

This work presented a real-time cutting simulation framework based on surface meshes enhanced with internal shape-preserving constraints (ISPCs). The method was evaluated on eight models of varying resolution and topology using single-cut, multiple-cut, and volume-preservation experiments. The results demonstrate stable, visually realistic cutting at interactive rates, with the incorporation of ISPCs preserving both volume and structural integrity under large deformations induced by gravity and collision.

The proposed GPU implementation substantially improves computational performance over our previous approaches. By parallelizing the triangle–triangle intersection stage and rendering directly from GPU-resident geometry buffers, the framework achieved frame-rate improvements of up to 677% over the slicing-based method and up to 395% over the CPU-based ISPC implementation. Crucially, the cutting stage itself is no longer a bottleneck: across the multiple-cutting experiments, its cost remained below 0.5 ms on average—approximately 1.3% of accumulated frame time even for the largest model—despite mesh growth of 55–295% after two cuts. Steady-state performance is in-

stead governed by deformation and inter-part collision handling, which together account for over 95% of frame time in all models except Armadillo. The Torus model illustrates this regime dependence most clearly: its total cost roughly tripled once the separated parts entered sustained contact, producing the bimodal frame-time distributions observed in the larger models.

The Armadillo model exposes the current scaling limit in two distinct ways. Its mean per-frame cost is dominated by the deformation and collision stages, which grow with vertex and triangle count; at the same time, its frame-time dispersion is driven by cutting, whose distribution is extremely heavy-tailed—the median cut cost is negligible, but rare cut-finalization frames, in which seam triangulation and constraint rebuilding execute on the CPU, produce the worst-case spikes that push the model below interactive rates. This indicates that further gains require both an optimized PBD solver and self-collision stage for high-resolution meshes and a migration of cut finalization off the critical path, for example, through asynchronous or incremental constraint rebuilding.

Volume preservation shows a similarly resolution- and geometry-dependent pattern. Simple-topology models (Sphere 20, Sphere 80, Torus, and Bunny) maintain volume to within 0.4%, confirming that non-trivial topology alone does not impair preservation. Liver, Banana, and Armadillo settle at post-collision plateaus of 92.0–93.3%. The Octopus model is the limiting case: despite constraint stiffness raised to 1, its averaged volume declines by approximately 27% with no recovery, stabilizing at 73.43% (averaged minimum of 72.97%). We attribute this to the ISPC generation strategy: inward ray casts from thin, highly curved features such as tentacles produce short or poorly oriented internal constraints that cannot resist compression, suggesting that geometry-aware constraint placement is required for complex shapes.

The proposed framework depends only on standard compute-shader functionality—structured buffers, thread-group shared memory, and atomic operations—exposed by Unity across Direct3D, Vulkan, and Metal back-ends, and therefore ports to any GPU architecture supported by these APIs without algorithmic changes. Performance scales primarily with the number of available compute units for the deformation, collision, and intersection stages, while the CPU-resident cut-finalization stage is architecture-independent. Memory requirements are fixed at initialization by the buffer pre-allocation policy, which reserves a constant multiple of the loaded mesh size; on GPUs with smaller VRAM, the pre-allocation multiple can be reduced at the cost of a lower bound on the number of supported successive cuts.

Future work will therefore focus on improving ISPC generation for geometrically complex objects, optimizing the deformation and self-collision stages for high-resolution meshes, incorporating more realistic material properties for surgical simulation, and validating simulation fidelity. Finally, because the cutting stage is only partially parallelized due to the complexity of dynamic mesh data structures, fully GPU-based cutting algorithms remain an important research direction.

Author Contributions: Conceptualization, L.K., Y.-J.C. and M.H.; methodology, L.K., Y.-J.C. and M.H.; software, L.K.; validation, L.K. and M.H.; formal analysis, Y.-J.C.; investigation, Y.-J.C. and L.K.; resources, L.K.; data curation, M.H.; writing—original draft preparation, L.K.; writing—review and editing, L.K. and M.H.; visualization, L.K.; supervision, M.H.; project administration, M.H.; funding acquisition, M.H. All authors have read and agreed to the published version of the manuscript.

Funding: This research study was supported by the Basic Science Research Program through the National Research Foundation of Korea (NRF), was funded by the Ministry of Education (NRF-2022R1I1A3069371), was funded by BK21 FOUR (Fostering Outstanding Universities for Research) (No.: 5199990914048), and was supported by the Soonchunhyang University Research Fund.

Data Availability Statement: The original contributions presented in this study are included in the article. Further inquiries can be directed to the corresponding author.

Conflicts of Interest: The authors declare no conflicts of interest.

Appendix A

Figure A1. Mean total simulation, deformation, collision and cutting time per sampled frame: (**a**) Sphere 20; (**b**) Sphere 80; (**c**) Torus; (**d**) Bunny; (**e**) Octopus; (**f**) Banana; (**g**) Liver; (**h**) Armadillo.

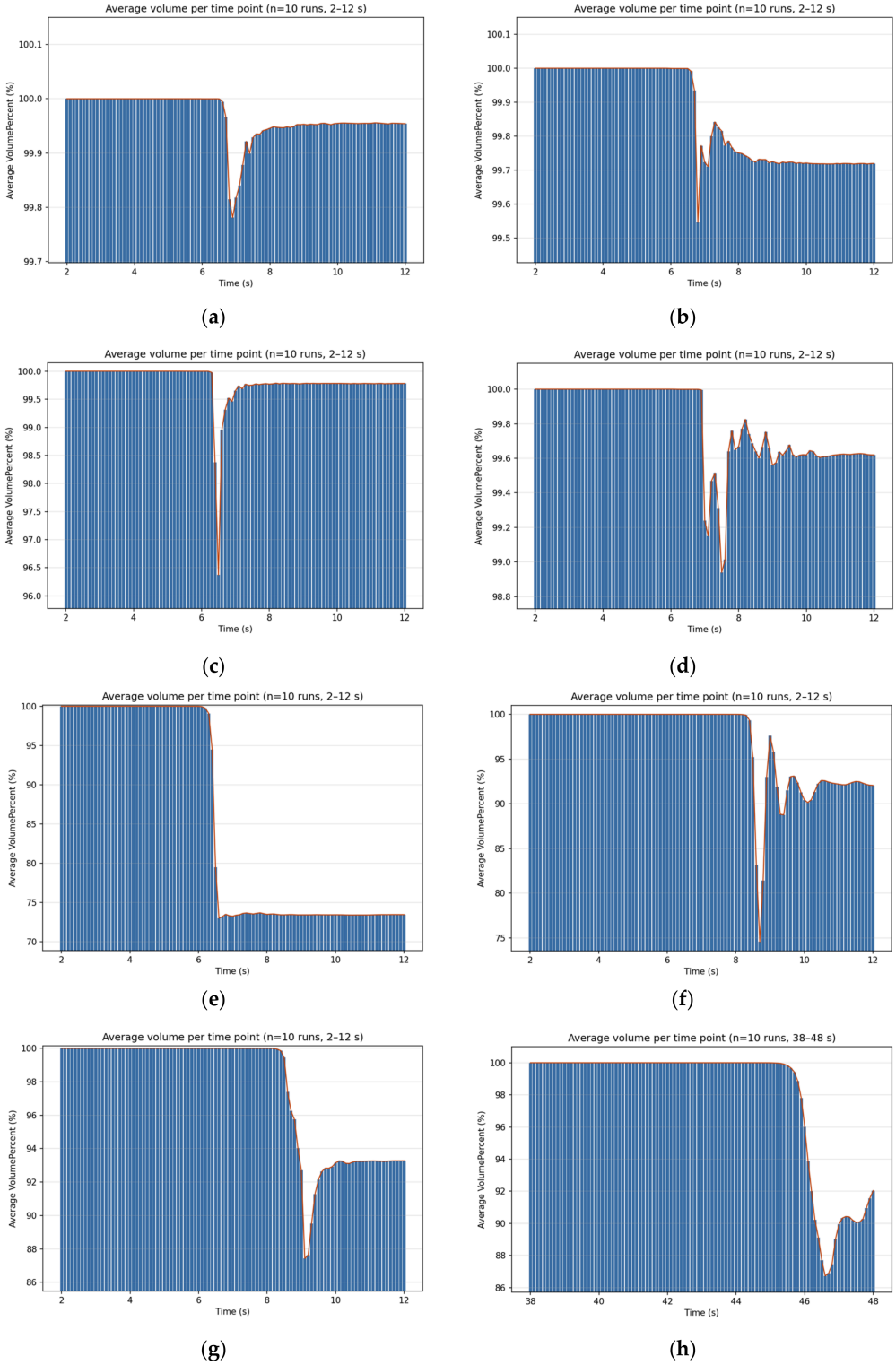

Figure A2. Histograms of mean volume per frame: (**a**) Sphere 20; (**b**) Sphere 80; (**c**) Torus; (**d**) Bunny; (**e**) Octopus; (**f**) Banana; (**g**) Liver; (**h**) Armadillo.

References

1. Portelli, M.; Bianco, S.F.; Bezzina, T.; Abela, J.E. Virtual reality training compared with apprenticeship training in laparoscopic surgery: A meta-analysis. *Ann. R. Coll. Surg. Engl.* **2020**, *102*, 672–684. [CrossRef]
2. Stepan, K.; Zeiger, J.; Hanchuk, S.; Del Signore, A.; Shrivastava, R.; Govindaraj, S.; Iloreta, A. Immersive virtual reality as a teaching tool for neuroanatomy. *Int. Forum Allergy Rhinol.* **2017**, *7*, 1006–1013. [CrossRef]
3. Imran, E.; Adanir, N.; Khurshid, Z. Significance of Haptic and Virtual Reality Simulation (VRS) in the Dental Education: A Review of Literature. *Appl. Sci.* **2021**, *11*, 10196. [CrossRef]

4. Riddle, E.W.; Kewalramani, D.; Narayan, M.; Jones, D.B. Surgical Simulation: Virtual Reality to Artificial Intelligence. *Curr. Probl. Surg.* **2024**, *61*, 101625. [CrossRef]
5. Wu, J.; Westermann, R.; Dick, C. A Survey of Physically Based Simulation of Cuts in Deformable Bodies. *Comput. Graph. Forum* **2015**, *34*, 161–187. [CrossRef]
6. Nienhuys, H.W.; Frank van der Stappen, A. A Surgery Simulation Supporting Cuts and Finite Element Deformation. In Proceedings of the Medical Image Computing and Computer-Assisted Intervention—MICCAI 2001, Utrecht, The Netherlands, 14–17 October 2001. [CrossRef]
7. Bielser, D.; Glardon, P.; Teschner, M.; Gross, M. A state machine for real-time cutting of tetrahedral meshes. *Graph. Model.* **2004**, *66*, 398–417. [CrossRef]
8. Steinemann, D.; Otaduy, M.A.; Gross, M. Fast arbitrary splitting of deforming objects. In *Proceedings of the 2006 ACM SIGGRAPH/Eurographics Symposium on Computer Animation (SCA '06)*; Eurographics Association: Goslar, Germany, 2006; pp. 63–72. [CrossRef]
9. Steinemann, D.; Harders, M.; Gross, M.; Szekely, G. Hybrid Cutting of Deformable Solids. In *Proceedings of the IEEE Conference on Virtual Reality (VR '06)*; IEEE Computer Society: Los Alamitos, CA, USA, 2006; pp. 35–42. [CrossRef]
10. Jia, S.; Zhang, W.; Yu, X.; Pan, Z. CPU–GPU Parallel Framework for Real-Time Interactive Cutting of Adaptive Octree-Based Deformable Objects. *Comput. Graph. Forum* **2018**, *37*, 45–59. [CrossRef]
11. Kim, J.-H.; Lee, J. Advanced GPU Techniques for Dynamic Remeshing and Self-Collision Handling in Real-Time Cloth Tearing. *IEEE Access* **2025**, *13*, 3188–3203. [CrossRef]
12. Byeon, S.P.; Lee, D.Y. Adaptive surface representation based on homogeneous hexahedrons for interactive simulation of soft tissue cutting. *Comput. Methods Programs Biomed.* **2021**, *200*, 105873. [CrossRef]
13. Lee, D.-K.; Kim, T.-W.; Choi, Y.-J.; Hong, M. Volumetric Object Modeling Using Internal Shape Preserving Constraint in Unity 3D. *Intell. Autom. Soft Comput.* **2022**, *32*, 1541–1556. [CrossRef]
14. Bender, J.; Müller, M.; Macklin, M. A survey on position based dynamics. In *Proceedings of the European Association for Computer Graphics: Tutorials (EG '17)*; Eurographics Association: Goslar, Germany, 2017; pp. 1–31. [CrossRef]
15. Müller, M.; Heidelberger, B.; Hennix, M.; Ratcliff, J. Position based dynamics. *J. Vis. Comun. Image Represent.* **2007**, *18*, 109–118. [CrossRef]
16. Wu, H.; Ye, F.; Gao, Y.; Cong, Y.; Hao, A. Real-Time Laparoscopic Cholecystectomy Simulation Using a Particle-Based Physical System. *Complex Syst. Model. Simul.* **2022**, *2*, 186–196. [CrossRef]
17. Huang, T.; Zhang, N.; Zhang, X.; Liao, H. Anatomy Education Method using Autostereoscopic 3D Image Overlay and Mid-Air Augmented Reality Interaction. *SID Symp. Dig. Tech. Pap.* **2022**, *53*, 91–94. [CrossRef]
18. Yu, P.; Zhao, Z.; Wang, R.; Pan, J. Real-time soft body dissection simulation with parallelized graph-based shape matching on GPU. *Comput. Methods Programs Biomed.* **2024**, *250*, 108171. [CrossRef]
19. Haiderbhai, M.; Gondokaryono, R.; Wu, A.; Kahrs, L.A. Sim2Real Rope Cutting With a Surgical Robot Using Vision-Based Reinforcement Learning. *IEEE Trans. Autom. Sci. Eng.* **2025**, *22*, 4354–4365. [CrossRef]
20. Khan, L.D.; Kim, T.; Hong, M. Real-Time Progressive Cutting of Deformable Objects in Unity 3D with Internal Shape Preserving Constraints. *Appl. Sci.* **2025**, *15*, 12984. [CrossRef]
21. Va, H.; Choi, M.-H.; Hong, M. Real-Time Cloth Simulation Using Compute Shader in Unity3D for AR/VR Contents. *Appl. Sci.* **2021**, *11*, 8255. [CrossRef]
22. Teschner, M.; Heidelberger, B.; Müller, M.; Pomerantes, D.; Gross, M.H. Optimized Spatial Hashing for Collision Detection of Deformable Objects. In Proceedings of the International Symposium on Vision, Modeling, and Visualization, Munich, Germany, 19–21 November 2003. Available online: https://cg.informatik.uni-freiburg.de/publications/2003_VMV_collisionDetectionHashing.pdf (accessed on 16 June 2026).
23. Meisters, G.H. Polygons Have Ears. *Am. Math. Mon.* **1975**, *82*, 648–651. [CrossRef]
24. Shewchuk, J.R. Delaunay refinement algorithms for triangular mesh generation. *Comput. Geom.* **2002**, *22*, 21–74. [CrossRef]
25. Lancaster, J.L.; Eberly, D.; Alyassin, A.; Downs, J.H., III; Fox, P.T. A geometric model for measurement of surface distance, surface area, and volume from tomographic images. *Med. Phys.* **1992**, *19*, 419–431. [CrossRef]
26. Khan, L.; Choi, Y.-J.; Hong, M. Cutting Simulation in Unity 3D Using Position Based Dynamics with Various Refinement Levels. *Electronics* **2022**, *11*, 2139. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.